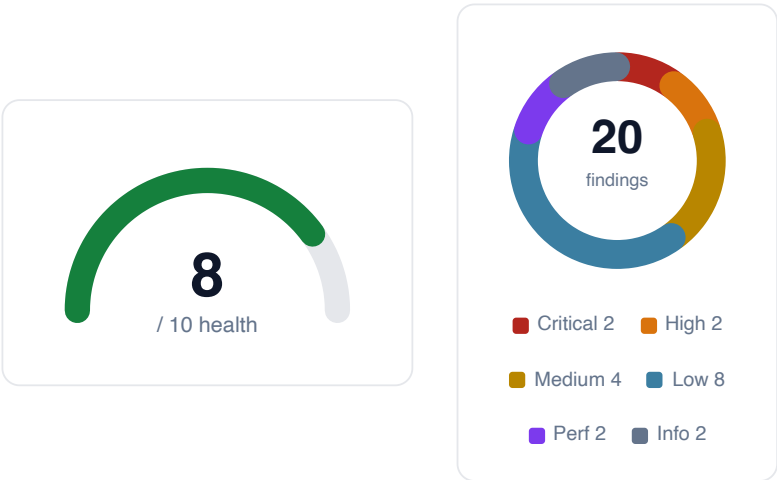


Security & Health Audit

ATC (self-audit — first real full ten-module engagement) — Multi-tenant Supabase/Next.js SaaS monorepo — apps/main (travel-agent platform) + apps/rag (embeddings/knowledge service), two Supabase projects
2026-07-17 · main @ 2be7c3d8 (working tree) · Prepared by Harvey

CONFIDENTIAL



TENANT ISOLATION

HOLDS — 416 RLS policies all gate on `auth_user_in_tenant(tenant_id); zero USING(true); JWT verification fail-closed. One latent cross-tenant WRITE gap in the tenantClient UPDATE path (no confirmed call site) is the only isolation finding.` verified

2 2 4 8
Critical High Medium Low

First full ten-module audit run against a REAL target (clears the never-run-on-real-target gap) and against BOTH monorepo apps + both Supabase DBs. Tenant isolation holds; no Critical or High SECURITY finding survived triage. The security signal is 6 semantic findings (3 Medium, 3 Low) — the notable one a latent cross-tenant WRITE via tenantClient's UPDATE path — plus a live-DB SECURITY DEFINER existence oracle (tenant_is_active). The mechanical secret scan's grade-F/12-Critical was fully triaged to false positives (test/doc placeholders; truffleshog verified 0). The real high-severity signal is in DATA CLASSIFICATION (M10: gmail_oauth_tokens and contacts hold Critical-class data; booking_passengers holds passport/DOB) and TEST QUALITY (M8: 27.8% mutation score with ~50% of code uncovered, and 53 tenant-isolation tests that mock the DB client so they cannot catch an RLS regression). Consistent with the June 2026-06-27 self-audit's 'tenant isolation holds' verdict; this run adds the tenantClient UPDATE finding and the two-app/live-DB coverage the prior run lacked.

Top bang-for-the-buck

80		RAG citation source_url rendered as an anchor href with no scheme validation — stored/DOM XSS
64		tenantClient UPDATE path does not sanitize tenant_id in the SET payload — latent cross-tenant write
60		gmail_oauth_tokens stores OAuth access/refresh tokens and API keys (Critical-class data)
60		SECURITY DEFINER RPC tenant_is_active is an existence/status oracle for arbitrary tenant IDs
60		Mechanical secret scan grade F is entirely false positives — 12 Critical hits are test/doc placeholders
51		Booking cancellation never claws back an 'available' (queued-but-not-transferred) payout

Action plan

Everything BFTB > 75, plus every Critical/High security finding.

#	ACTION	WHY	BFTB	OWNER
---	--------	-----	------	-------

1	Confirm the service-role-only lock (verified today), ensure tokens are encrypted at rest / stored in a vault rather than plaintext columns, enforce refresh rotation + revocation on disconnect, and add access auditing. Table is empty now — get this right before the Gmail integration ships (currently 501/gated).	gmail_oauth_tokens stores OAuth access/refresh tokens and API keys (Critical-class data) — Critical	60	Eng
2	No action on the data itself (legitimately held). Prioritize real (non-mocked) RLS regression tests on contacts and the CRM read paths; confirm retention/export/delete (CCPA) flows cover it.	contacts holds customer PII (names, emails, phones) — Critical sensitivity weighting — Critical	48	Eng
3	Confirm tenant RLS + service-role locks on each (spot-verified clean today); review the 4 JSONB container columns for nested PII; ensure passport/DOB/payment fields are covered by retention + encryption posture and by real isolation tests.	booking_passengers holds passport numbers + DOB; tenants holds payment refs (PCI); forensics_log/trip_* hold secrets — High	48	Eng
4	Add real RLS regression tests that run against a seeded two-tenant local stack (the M2 dynamic tier) and assert tenant A cannot read/write tenant B's rows — not mocked-client unit tests. This is also why M2 (dynamic) should be stood up.	53 tenant-isolation tests mock the DB client — they cannot catch an RLS/tenant regression — High	40	Eng
5	Validate the scheme at render (allow only http/https) or sanitize at ingestion; reuse the app's existing safe-redirect/URL parsing helper.	RAG citation source_url rendered as an anchor href with no scheme validation — stored/DOM XSS — Medium	80	Operator

Scope & methodology

Reviewed Both monorepo apps. apps/main: 269 API routes, 22 auth-lib + db-layer files, proxy.ts, money/Al/admin surfaces (5 parallel semantic passes). apps/rag: 24 routes, 59 src files, vector store (1 semantic pass). Mechanical tiers (M4/M5/M6/M7-code/M8-intent/M9) over the whole repo root. Connected tier: live security+performance advisors and live PII classification against BOTH Supabase projects (main mfaknjyqiwcjokucnea, rag jjznkprbotkqquvcost), read-only. Live DETECT-DEEPER on the 3 flagged SECURITY DEFINER functions and all RLS-enabled-no-policy grants.

Tooling src/cli tools run directly (the operator module workflow) rather than the run-audit orchestrator, chiefly because the orchestrator's M5 (knip) probe hangs on ATC's full monorepo workspace resolution. Mechanical: quick-scan (M1), quality-scan (M4/M5), detect-static (M6/M7/M8/M9), pii-classify (M10). Semantic M1: 6 parallel review subagents (5 over apps/main, 1 over apps/rag) applying docs/scan-extras.txt + docs/fp-rules.txt, each with an explicit verified-safe section. Connected: Supabase advisors + live pii-classify via MCP/DB-URL on BOTH projects; SECURITY DEFINER bodies read via pg_get_functiondef and RLS/grants via role_table_grants on both live DBs (verdicts measured, not recalled). M8 mutation: a fresh scoped StrykerJS run (auth/audit/api routes, ~20k mutants, ~15 min) plus ATC's own whole-repo Jun-17 report; a fresh whole-repo run is tens of minutes (not run this session).

Module coverage

All ten audit modules, and whether each was assessed on this engagement. A module marked **Not assessed** produced no findings because it did not run — **this is not a clean result**; the reason is stated.

MODULE	AREA	STATUS	WHAT RAN / WHY NOT
M1	Multi-tenant security (lead)	Assessed	mechanical (quick-scan, grade F → 12 Critical secrets all triaged to test/doc-placeholder false positives; trufflehog 0 verified) + semantic (6 parallel review passes over apps/main + apps/rag, docs/scan-extras.txt lens) + connected/live (advisors both DBs; 3 SECURITY DEFINER bodies + all RLS-no-policy grants adjudicated on both live DBs; RAG vector-store isolation + RS256 JWT verifier verified live)
M2	Local pen-test (dynamic)	Not assessed	No two-tenant local Supabase stack stood up this run. dynamic-validate returned NO-GO: it searches <target>/supabase/migrations, which is empty in ATC's monorepo layout (real migrations live under apps/main/supabase/migrations and apps/rag/supabase/migrations). Operator/live work — see #159/#161; harness monorepo-path gap noted.
M3	Hotspot analysis	Not assessed	vitals_cli.py (external plugin) is not on PATH in this environment, so hotspot-scan could not produce the M3 ranked table. A .vitals/store.db exists but no capture was replayable this run.
M4	Duplication	Partial	jscpd RAN over apps/main (400 duplication clones + 23 diverged-clone near-misses). Scoped to apps/main because the whole-monorepo quality-scan hung on the knip stage; apps/rag duplication not separately censused.
M5	Slop / dead code	Partial	BOTH tiers ran, but on apps/main scope: the static M5-slop tier via detect-static whole-repo (505 single-use-helper + unused imports/narrating comments/orphan TODOs), and knip dead-code via quality-scan on apps/main (840 findings). The whole-MONOREPO knip hung on workspace resolution, so apps/rag's knip dead-code tier was not separately run — hence partial, not ran.
M6	Simplification / maintainability	Assessed	
M7	Performance	Assessed	code tier (detect-static: 86 findings — await-in-loop N+1 x45, client fetch in useEffect x22, state sprawl x16) + connected DB advisors on BOTH projects (main: 214 unused indexes / 5 unindexed FKs; rag: unused indexes + 4 unindexed FKs + 1 WARN auth_rls_initplan on rag_media_assets).
M8	Test quality	Assessed	test-intent tier (detect-static: 89 findings — 53 tenant-isolation tests mock the DB client, 21 call-count-only, 9 assertion-free, 3 happy-path-only on security-critical) + mutation score from ATC's own Stryker reports: 27.8% whole-repo (Jun-17 broad) and 29.4% on a fresh Jul-17 auth/audit/api-route scoped run; ~50% of mutants sit on code with no test coverage.
M9	App Router boundary/rendering	Assessed	detect-static: 76 findings — data-fetching waterfall x54, SSR-only API misuse x15, accidental dynamic rendering x5, unsafe/missing cache config x2.
M10	Data classification (PII/PHI/PCI)	Assessed	live classification against BOTH DBs (main: 50 findings incl. 2 Critical / 7 High; rag: 2 Low) + schema tier over both apps' migrations. Live rows sampled read-only.

Out of scope M2 dynamic pen-test: requires-live-run — dynamic-validate returned NO-GO because it looks for top-level supabase/migrations (ATC's live under apps/{main,rag}/supabase/migrations), and a two-tenant local stack was not stood up this run. M3 hotspot analysis: requires-live-run — vitals_cli.py is not on PATH in this environment. M7 Lighthouse/Core-Web-Vitals not run this session. M5 knip dead-code did not complete on the full monorepo (hung on workspace resolution); the static M5-slop indicator tier did run and a scoped apps/main knip was launched.

Findings

M10-01	gmail_oauth_tokens stores OAuth access/refresh tokens and API keys (Critical-class data)	Critical	BFTB 60	Review
Data classification · M10 — Secret/credential store · apps/main DB — public.gmail_oauth_tokens (live, 0 rows currently) · Open — data-classification disclosure · V5-E3-S5				
Evidence pii-classify (live) flagged AUTH_TOKEN + API_KEY column classes. RLS enabled, service-role-only grants (verified deny-all to anon/authenticated).				
Impact Compromise of this table = takeover of connected Gmail accounts (read customer mail, send as tenant). Highest-sensitivity store in the schema; must have the strictest access + encryption-at-rest posture and short-lived tokens.				
Fix Confirm the service-role-only lock (verified today), ensure tokens are encrypted at rest / stored in a vault rather than plaintext columns, enforce refresh rotation + revocation on disconnect, and add access auditing. Table is empty now — get this right before the Gmail integration ships (currently 501/gated).				
When this is OK vs. not — confirm against your design: ✓ OK when Service-role-only (confirmed), tokens encrypted/vaulted, rotation + revocation implemented. ✗ Not OK when Tokens stored plaintext, or any client-role grant is added later.				
M10-02	contacts holds customer PII (names, emails, phones) — Critical sensitivity weighting	Critical	BFTB 48	Review
Data classification · M10 — PII/SENSITIVE_PII store · apps/main DB — public.contacts (live, 9 rows) · Open — data-classification disclosure · V4-E3-S5				
Evidence pii-classify (live): NAME, EMAIL, PHONE classes. RLS enabled with tenant policies (contacts is a tenant-scoped table).				
Impact Core CRM PII. A tenant-isolation regression here is a direct customer-data breach — which is why the M8 finding that tenant-isolation tests mock the DB client (M8-01) matters most on exactly these tables.				
Fix No action on the data itself (legitimately held). Prioritize real (non-mocked) RLS regression tests on contacts and the CRM read paths; confirm retention/export/delete (CCPA) flows cover it.				
When this is OK vs. not — confirm against your design: ✓ OK when Tenant RLS enforced (verified) AND covered by tests that exercise real RLS, not a mocked client. ✗ Not OK when Isolation is only asserted by tests that mock the DB client (current state — see M8-01).				
M10-03	booking_passengers holds passport numbers + DOB; tenants holds payment refs (PCI); forensics_log/trip_* hold secrets	High	BFTB 48	Review
Data classification · M10 — Sensitive-PII / PCI / secret stores · apps/main DB — public.{booking_passengers (PASSPORT, DOB, NAME), tenants (PAYMENT_REF, EMAIL, PHONE, ADDRESS), forensics_log (API_KEY), trip_itineraries/trip_resources (AUTH_TOKEN), tenant_branding (EMAIL, API_KEY), host_adapters (HEALTH), auth_attempts (IP, EMAIL)} · Open — data-classification disclosure · V4-E3-S5				
Evidence pii-classify (live) — 7 High-severity tables. Plus JSONB 'review for nested PII' flags on gmail_inbound_messages.raw_payload, booking_line_items.item_details, inbound_emails.raw_payload, weather_forecast_cache.payload.				
Impact Passport + DOB is sensitive-PII (travel/identity-theft exposure); payment refs are PCI-adjacent; stored API keys/auth tokens are secret-class. These raise the blast radius of any isolation or access-control slip and drive breach-notification obligations.				
Fix Confirm tenant RLS + service-role locks on each (spot-verified clean today); review the 4 JSONB container columns for nested PII; ensure passport/DOB/payment fields are covered by retention + encryption posture and by real isolation tests.				

M8-01

53 tenant-isolation tests mock the DB client — they cannot catch an RLS/tenant regression

HighBFTB 40Confirmed

Test quality · M8 — Tenant-isolation test mocks the DB client · `apps/main/test/unit/{auth/ssr-client.test.ts, db/tenant-client.test.ts, db/platform-admin-audit.test.ts, ...}` (53 sites) · Open · V5·E2·S5

Evidence

detect-static test-intent tier: 53 High-severity findings where tests asserting tenant isolation mock the Supabase/DB client, so the assertion exercises the mock's behavior, not real RLS. A test that can't fail when the isolation logic breaks is not protecting the isolation.

Impact

The exact tables that hold Critical/High PII (contacts, booking_passengers) are 'covered' by tests that would stay green through an RLS regression. This is the highest-value test-quality gap given the M10 data sensitivity, and it undercuts the confidence the RLS audit otherwise earns.

Fix

Add real RLS regression tests that run against a seeded two-tenant local stack (the M2 dynamic tier) and assert tenant A cannot read/write tenant B's rows — not mocked-client unit tests. This is also why M2 (dynamic) should be stood up.

M1-02

RAG citation source_url rendered as an anchor href with no scheme validation — stored/DOM XSS

MediumBFTB 80Likely

Security · M1 — Stored XSS (untrusted URL) · `apps/main/src/components/chat/MessageSources.tsx:55` · Open · V4·E5·S5

Evidence

A RAG citation's source_url (user-ingested during knowledge ingestion, never scheme-validated) is rendered as `<a href={source_url}>`. A javascript: or data: URL executes on click.

Impact

Intra-tenant stored/DOM XSS: a poisoned ingested document's citation URL runs script in a coordinator's browser on click. Gated by ingestion + a click, so Medium.

Fix

Validate the scheme at render (allow only http/https) or sanitize at ingestion; reuse the app's existing safe-redirect/URL parsing helper.

When this is OK vs. not — confirm against your design:

✓ OK when source_url is scheme-validated (http/https only) at ingestion or render.

✗ Not OK when Raw user-ingested URL is placed in href (current state).

M1-01

tenantClient UPDATE path does not sanitize tenant_id in the SET payload — latent cross-tenant write

MediumBFTB 64Likely

Security · M1 — Tenant isolation (write) · `apps/main/src/lib/db/tenant-client.ts:90` · Open · V5·E4·S4

Evidence

The tenantClient Proxy overrides tenant_id on insert/upsert (hardening the payload), but on UPDATE it only appends the `.eq('tenant_id', ctx.tenant_id)` WHERE filter and passes the SET payload through untouched. A handler that spreads untrusted input into `.update()` on a tenant-scoped table could set tenant_id to an attacker-named value, reassigning the row OUT of its tenant.

Impact

Cross-tenant WRITE (row hand-off) if any update handler forwards client-controlled fields including tenant_id. The WHERE filter guarantees you can only update your OWN rows, but nothing stops the new tenant_id from pointing elsewhere. Mechanism certain; no confirmed exploitable call site was found in scope (hence Medium/Likely).

Fix

In the UPDATE branch of the proxy, strip/override tenant_id in the SET payload the same way insert/upsert already do (defense-in-depth, one place). Optionally add a DB-level WITH CHECK on tenant-scoped update policies so the write is rejected server-side too.

When this is OK vs. not — confirm against your design:

✓ OK when Every update call site passes an explicit whitelist that excludes tenant_id.

✗ Not OK when Any handler spreads req.body into `.update()` on a tenant-scoped table.

M1-03

Booking cancellation never claws back an 'available' (queued-but-not-transferred) payout

MediumBFTB 51Likely

Correctness · M1 — Money integrity · `apps/main/src/app/api/bookings/[id]/cancel/route.ts:223-332` · Open · V4·E4·S4

Evidence

'available' payout rows always have `stripe_transfer_id == null` (set only at paid-settle), so the Stripe-reversal branch is skipped and the handler returns `no_action`. The route header claims 'available → reversal', which the code does not do; the next transfer cron then pays the sub-host for a cancelled booking.

Impact

Platform-side money leak: a sub-host can be paid for a cancelled booking when the payout was queued-but-not-yet-transferred at cancel time. Not cross-tenant. Reachability depends on commission state at cancel time.

Fix

In the cancel handler, transition 'available' payouts to a cancelled/withheld state (not just the transferred→reversal path) so the transfer cron skips them; add a test for cancel-while-available.

M8-02

Mutation score 27.8% whole-repo (29.4% on the auth/audit/API scope); ~50% of code has no test coverage

MediumBFTB 32Confirmed

Test quality · M8 — Mutation / coverage · apps/main + apps/rag (StrykerJS; ATC's own reports, whole-repo Jun-17 + fresh auth/audit/api Jul-17) · Open · V4-E2-S5

Evidence

Overall 52,152 mutants: 14,119 killed, 10,237 survived, 26,488 no-coverage → 27.8%. Fresh scoped run (263 files, auth/audit/api routes): 29.4%. Denial/boundary survivors concentrate on security-adjacent lib: sign-service-jwt.ts (12/19), pii-scrubber.ts (13/23), billing/payment-state.ts (4/7), memory/dob.ts (13/15), rag-auth. Caveat: the mutate scope includes API routes ATC tests via e2e (not unit), so no-coverage overstates true untested-ness for the route layer.

Impact

Half the mutated code has no test able to detect a behavior change; boundary/negation logic in security-adjacent files (JWT signing, PII scrubbing, payment state, DOB handling) is where survivors cluster — a class of silent-regression risk.

Fix

Add boundary/negation tests to the flagged lib files (start with sign-service-jwt, pii-scrubber, payment-state, dob); wire the e2e suite into the mutation coverage picture so route-layer 'no-coverage' reflects reality.

M1-04

SECURITY DEFINER RPC tenant_is_active is an existence/status oracle for arbitrary tenant IDs

LowBFTB 60Confirmed

Security · M1 — SECURITY DEFINER exposure · apps/main DB — public.tenant_is_active(uuid), exposed via /rest/v1/rpc/tenant_is_active · Open · V3-E5-S5

Evidence

Body: SELECT EXISTS(SELECT 1 FROM public.tenants WHERE id=target_tenant_id AND status='active') — no auth.uid()/caller check, unlike its two siblings (auth_user_in_tenant, auth_user_can_access_conversation) which are both caller-scoped and were verified safe. EXECUTE is granted to authenticated.

Impact

Any signed-in user can confirm whether an arbitrary tenant UUID exists and is active. Low practical exploitability (tenant IDs are unguessable UUIDs) but a least-privilege violation on the exposed API.

Fix

REVOKE EXECUTE ON FUNCTION public.tenant_is_active(uuid) FROM authenticated (RLS policies calling it as definer are unaffected), or switch to SECURITY INVOKER, or add an auth.uid() membership predicate.

When this is OK vs. not — confirm against your design:

✓ OK when Only RLS policies call it (as definer) — then revoke EXECUTE from authenticated.

✗ Not OK when Clients can call the RPC directly (current state).

M1-08

Mechanical secret scan grade F is entirely false positives — 12 Critical hits are test/doc placeholders

LowBFTB 60Confirmed

Security · M1 — Secret exposure (triaged) · apps/main/test/**, tests/**, docs/runbooks/supabase-setup.md · Triaged — recommend scanner allowlist · V3-E5-S5

Evidence

quick-scan graded ATC F on 12 Critical 'committed credential' findings (from 244 raw gitleaks working-tree hits). All 12 are in tracked test/fixture/doc files; sampled values are placeholders (example.com, <project-ref>, abcxyz, a redact-secrets.test.ts fixture). trufflehog (which verifies) found 0 verified and 0 unverified secrets. Real .env.local/*.pem are gitignored, not committed.

Impact

No real exposure. The value here is the triage itself: the free-tier grade F would alarm a client; the paid triage shows it's noise — the product's free→paid precision story on a real repo.

Fix

Add a gitleaks allowlist for test/fixture/runbook-doc paths (or example.com/<project-ref>/abcxyz patterns) so the grade reflects real risk and a genuine committed secret would stand out. Spot-check the ~unsampled hits to close the loop.

M1-09

RAG rag_retrieval_log_daily has a permissive USING(TRUE) RLS policy (latent — grant layer currently denies)

LowBFTB 48Confirmed

Security · M1 — Permissive RLS (latent) · apps/rag/supabase/migrations/...0017:26 — public.rag_retrieval_log_daily · Open · V3-E4-S5

Evidence

The rag_retrieval_log_daily RLS policy is USING(TRUE). Verified live: anon/authenticated/public hold ZERO table grants on every RAG table, so the grant layer denies before RLS runs — no live cross-tenant exposure today. It is the one permissive policy in the RAG DB where a single stray GRANT would immediately leak cross-tenant retrieval analytics with no second layer to catch it.

Impact

No live leak (grant layer denies). Latent defense-in-depth gap: a future GRANT SELECT to a client role on this table would expose all tenants' retrieval-analytics rows.

Fix

Replace USING(TRUE) with a tenant predicate (tenant_id = auth_user_tenant or the RAG JWT tenant claim), so isolation does not rest solely on the absence of a grant.

When this is OK vs. not — confirm against your design:

✓ OK when The table never receives a client-role grant (current state).

✗ Not OK when Any anon/authenticated SELECT grant is added to rag_retrieval_log_daily.

M1-05

Vestigial client SELECT grant on RLS-locked reference tables (pricing_seat_ladder, tier_definitions)

LowBFTB 40Confirmed

Security · M1 — RLS grant/policy mismatch · apps/main DB — public.pricing_seat_ladder, public.tier_definitions · Open · V2-E5-S5

Evidence Both have RLS enabled with NO policy (deny-all) yet still grant SELECT to anon+authenticated. RLS denies regardless, so the grant is vestigial; a client PostgREST read silently returns empty. Both documented as service-role-only reference data.

Impact Not a data leak (no rows exposed). Hygiene / least-privilege, and a latent 'silently-broken client read' if a feature ever tries to read them via the anon/authenticated key.

Fix REVOKE SELECT ... FROM anon, authenticated (service-role-only), or add an explicit read policy if client reads are actually intended.

M1-06

Customer quote-access token stored and compared in plaintext (hashed elsewhere)

LowBFTB 38Likely

Security · M1 — Token at rest · apps/main/src/app/api/quotes/[id]/send/route.ts:61 (quotes.customer_access_token) · Open · V3-E4-S4

Evidence The public quote-accept token is 256-bit random but stored/compared in plaintext, while the codebase's PAT and public-chat tokens are SHA-256 hashed at rest.

Impact Not guessable; risk only materializes under a read-only DB/backup breach (then the token is directly usable). Defense-in-depth / consistency gap.

Fix Hash the quote-access token at rest (match the PAT pattern): store SHA-256, compare hashes.

M1-10

RAG service-JWT verifier tolerates absent iss/aud claims during a documented rollout

LowBFTB 32Review

Security · M1 — JWT validation (hardening) · apps/rag/src/lib/auth/verify-service-jwt.ts:157-187 · Open (issue-tracked exception #1773/#1843) · V2-E4-S5

Evidence The verifier pins RS256, checks exp/iat(±5min), kid allowlist, and Redis jti replay, but tolerates absent iss/aud during a documented signing rollout. Signature is the hard gate.

Impact Residual token-confusion risk only if the RS256 signing key is shared with another service that omits aud — otherwise none. Appendix-grade.

Fix Once the rollout completes, make iss/aud mandatory (remove the tolerance branch) so a shared-key confusion attack is impossible by construction.

M6-01

Maintainability: hand-rolled base64url + 11x duplicated email regex + scattered currency formatting

LowBFTB 32Review

Maintainability · M6 — Reinvention (verdict) · apps/main (base64url: lib/privacy/customer-hash.ts:27, webhooks/gmailpubsub/route.ts:298; email regex: 11 route handlers; currency: 23 display sites) · Open (rollup — first M6 verdict on a real target) · V2-E4-S5

Evidence 52 hand-rolled indicators triaged: genuine — base64url manual replace-chain where Node's .toString('base64url') exists (2, one in a security-adjacent PII hash); the same email regex copy-pasted 11x; manual \${cents/100} currency formatting at 23 sites (partial — several have bespoke rounding Intl can't do). Non-actionable/suppressed: cookie header strings (idiomatic), random-string-id (all in test code), class-merge, deep-equal.

Impact Consistency/maintainability, no correctness or security risk. No reinvented crypto/auth/tenancy primitive was found (those hand-rolled security primitives — constant-time-equal, hmac-jwt, postgres-filter escaping — were reviewed by the M1 core-isolation pass and judged sound).

Fix Use buf.toString('base64url'); extract one shared email validator (or Zod .email()); centralize a formatCents() helper preserving intentional rounding.

M1-07

tenantClient service-role passthrough surfaces (PLATFORM_READABLE_TABLES; .storage/.functions/.rest) rely on caller self-scoping

Low

BFTB 29

Review

Security · M1 — Tenant isolation (defense-in-depth) · `apps/main/src/lib/db/tenant-client.ts:144, 153` · [Open](#) · V3-E3-S4

Evidence PLATFORM_READABLE_TABLES passthrough is single-layer service-role with no RLS backstop (email_log, nullable tenant_id, sits in that set relying on caller self-scoping); the top-level proxy blocks only rpc/schema, leaving .storage/.functions/.rest as raw unscoped service-role surfaces.

Impact No confirmed leak — these are used by trusted server code today. But they are the paths where a future careless caller would bypass tenant scoping with no second layer to catch it.

Fix Add an RLS backstop or an explicit tenant filter on the platform-readable set (esp. email_log); consider narrowing/scoping the .storage/.functions/.rest passthrough or documenting the required caller discipline.

When this is OK vs. not — confirm against your design:

✓ **OK when** Only trusted server code reaches these and applies its own tenant filter.

✗ **Not OK when** An untrusted-input-reachable caller uses the passthrough without self-scoping.

M7-01

DB advisor debt: 214 unused indexes + 5 unindexed FKs (main), plus a per-row RLS initplan on rag_media_assets (rag)

Perf

BFTB 29

Confirmed

Performance · M7 — DB advisors · `main DB (mfaknjyqiwcjojukcnea) + rag DB (jjznkprbotkqgnuvcost)` · [Open](#) · V3-E4-S3

Evidence main: 214 unused_index (89 tables) + 5 unindexed_foreign_keys on email_retry_content (incl. tenant_id). rag: ~20 unused_index + 4 unindexed FKs + 1 WARN auth_rls_initplan on rag_media_assets ('rag_media_assets_select re-evaluates auth.<function>() per row').

Impact Write-amplification + storage debt (unused indexes) and slow FK joins (email_retry_content, incl. the RLS-relevant tenant_id FK). The rag_media_assets initplan degrades that policy's query at scale.

Fix Add covering indexes on email_retry_content FKs (esp. tenant_id); rewrite the rag_media_assets policy to use (select auth.<function>()) per the Supabase initplan guidance; review the unused-index list for removal after confirming they aren't reserved for planned queries.

M9-01

App Router rendering/data-fetching hygiene: 76 findings led by 54 data-fetching waterfalls

Perf

BFTB 29

Likely

Performance · M9 — App Router boundary/rendering · `apps/main + apps/rag (detect-static M9 AST)` · [Open \(rollup\)](#) · V3-E3-S4

Evidence 76 M9 findings: data-fetching waterfall x54 (sequential awaits that could be parallel), SSR-only API misuse x15, accidental dynamic rendering x5 (pages opting out of static that don't need per-request data), unsafe/missing cache config x2.

Impact Latency (serial fetches), unnecessary dynamic rendering (lost caching/perf), and a few SSR/browser-API boundary mismatches. Hygiene/perf, not security.

Fix Parallelize independent awaits (Promise.all) on the 54 waterfall sites; audit the 5 accidental-dynamic pages for static eligibility; fix the SSR-only API misuse sites. Prioritize by route traffic (M3 hotspots would rank these — deferred).

M5-01

Dead code / slop: 840 knip dead-code findings (apps/main) + 505 static single-use-helper indicators

Info

BFTB 19

Review

Slop / dead code · M5 — Dead code + slop indicators · `apps/main (knip) + whole repo (detect-static M5-slop tier)` · [Open \(rollup — apps/rag knip not separately run\)](#) · V2-E3-S4

Evidence knip on apps/main: 840 dead-code findings (unused exports/files/deps). detect-static M5-slop (whole repo): 505 single-use-helper, 5 unused-import, 4 narrating-comment, 3 single-call-wrapper, 2 orphan-TODO, 1 each else-after-return/rethrow-catch/Al-comment-phrasing. Whole-monorepo knip hung; apps/rag's knip dead-code tier not separately run.

Impact Maintainability debt. 840 knip items on apps/main is a large dead-code surface (unused exports/files) worth a cleanup sweep; the single-use-helper count is Info by design. apps/rag dead-code census incomplete — fail-loud, not clean.

Fix Run knip per-app (apps/rag next); triage the 840 apps/main items (many are likely safe deletions — knip is high-precision on unused exports); review single-use-helpers for inlining.

M4-01 Duplication: 400 jscpd clones + 23 diverged-clone near-misses (apps/main)

[Info](#)[BFTB 19](#)[Review](#)

Duplication · M4 — Duplication · `apps/main (jscpd + diverged-clone pass)` · Open (rollup — apps/rag not separately censused) · V2·E3·S4

Evidence quality-scan on apps/main: 400 duplication clones + 23 diverged-clone near-misses (the #360 tenant-key/supabase-query heuristic pass). Test-file clones are not elevated to security severity (#400).

Impact Maintainability: duplicated logic drifts. The 23 diverged clones matter more than raw count — near-identical blocks that have started to diverge are where a fix applied to one copy silently misses the others (relevant for the 11× duplicated email regex in M6-01).

Fix Prioritize the 23 diverged clones for consolidation; sample the 400 exact clones for extract-shared-helper candidates. Run jscpd on apps/rag to complete the census.